

USB Device Monitoring Tool

Python-Based USB Connection and Disconnection Monitoring with Windows Notifications

1. Project Overview

This project implements a Python-based USB Device Monitoring Tool that continuously checks the system for USB devices. It detects when a USB device is connected or removed, identifies the device using its Vendor ID and Product ID, displays the event in Command Prompt, and generates a Windows desktop notification.

The project was developed and tested on a Windows laptop. The monitoring logic uses PyUSB with the libusb backend, while win10toast is used to display Windows notifications.

2. Project Objectives

- Detect USB devices connected to the Windows system.
- Detect when a previously detected USB device is disconnected.
- Capture the Vendor ID and Product ID of detected devices.
- Display connection and disconnection events in Command Prompt.
- Generate a Windows desktop notification for each connection or disconnection event.
- Continuously monitor the system without requiring manual scanning.

3. Project Structure

The project folder was created on the Desktop with the following structure:

```
USB-Device-Monitor
|
├── evidence
├── logs
├── screenshots
├── src
│   └── usb_monitor.py
└── README.md
```

The main Python script is stored in the src folder as usb_monitor.py. The other folders can be used for evidence, logs, screenshots, and project documentation.

4. Opening the Project Folder

The project was accessed through Windows File Explorer and Command Prompt. Opening Command Prompt from the project folder ensures that Python commands are executed in the correct location.

```
C:\Users\Lenovo\Desktop\USB-Device-Monitor>
```

The src directory was then opened:

```
cd src
```

The contents of the src directory were checked with:

```
dir
```

The main file identified was:

```
usb_monitor.py
```

5. Required Python Packages

The project uses the following Python components:

- PyUSB for USB device access.
- libusb1 backend for communication with USB devices.
- libusb_package to locate the libusb library.
- win10toast to display Windows desktop notifications.

6. Installing the Notification Package

From the src folder, the notification package was installed or verified with:

```
pip install win10toast
```

The system reported that win10toast was already installed. The package was therefore ready to use.

The installation can also be checked using:

```
python -m pip install win10toast
```

7. Existing USB Monitoring Logic

The original monitoring logic uses PyUSB to obtain the currently connected USB devices. Each device is represented by a tuple containing its Vendor ID and Product ID.

The script stores the previous device list, scans the system again after a short delay, and compares the new list with the previous list.

- Current device minus previous device = newly connected devices.
- Previous device minus current device = disconnected devices.
- The comparison repeats every two seconds.

8. Adding Windows Notifications

The win10toast library was added to the Python script using:

```
from win10toast import ToastNotifier
```

A notification object was then created:

```
notifier = ToastNotifier()
```

When a device is connected, the script sends a notification containing the Vendor ID and Product ID. When a device is removed, a similar notification is generated.

9. Final Python Script

The final version below includes USB monitoring, Command Prompt output, and repeated Windows notifications. The threaded notification option was intentionally removed so that repeated connect and disconnect notifications worked reliably during testing.

```
import time
import usb.core
import usb.backend.libusb1
import libusb_package
from win10toast import ToastNotifier

def get_backend():
    return usb.backend.libusb1.get_backend(
        find_library=lambda x: libusb_package.find_library(x)
    )

def get_usb_devices():
    backend = get_backend()
    devices = usb.core.find(find_all=True, backend=backend)

    if devices is None:
        return set()

    return {
        (device.idVendor, device.idProduct)
        for device in devices
    }

notifier = ToastNotifier()

previous_devices = get_usb_devices()

print("USB Device Monitor Started...")
print("Monitoring USB devices...")
print("Press Ctrl+C to stop.\n")

while True:
    current_devices = get_usb_devices()

    connected = current_devices - previous_devices
    disconnected = previous_devices - current_devices

    for vendor_id, product_id in connected:
        print(
            f"[CONNECTED] Vendor ID: {vendor_id:04x}, "
            f"Product ID: {product_id:04x}"
        )

        notifier.show_toast(
            "USB Device Connected",
            f"Vendor ID: {vendor_id:04x}\nProduct ID: {product_id:04x}",
            duration=3
        )
```

```

    )

    for vendor_id, product_id in disconnected:
        print(
            f"[DISCONNECTED] Vendor ID: {vendor_id:04x}, "
            f"Product ID: {product_id:04x}"
        )

        notifier.show_toast(
            "USB Device Disconnected",
            f"Vendor ID: {vendor_id:04x}\nProduct ID: {product_id:04x}",
            duration=3
        )

    previous_devices = current_devices

    time.sleep(2)

```

10. Saving the Python File

The final code was saved in the following location:

```
C:\Users\Lenovo\Desktop\USB-Device-Monitor\src\usb_monitor.py
```

When using Notepad, the file must be saved as a Python file with the .py extension. The first line of the file should begin with import time, and the last line should be time.sleep(2).

11. Running the USB Monitor

From the src directory, the script is started with:

```
python usb_monitor.py
```

A successful startup displays:

```

USB Device Monitor Started...
Monitoring USB devices...
Press Ctrl+C to stop.

```

The script continues running and checks for changes every two seconds.

12. Command Prompt Output

When a USB device is connected, the Command Prompt displays an event similar to:

```
[CONNECTED] Vendor ID: 22d9, Product ID: 2764
```

When the device is removed, the Command Prompt displays:

```
[DISCONNECTED] Vendor ID: 22d9, Product ID: 2764
```

The actual Vendor ID and Product ID depend on the USB device being tested.

13. Windows Desktop Notification

In addition to the Command Prompt output, the tool displays a Windows desktop notification.

For a connection event:

```
USB Device Connected  
Vendor ID: 22d9  
Product ID: 2764
```

For a disconnection event:

```
USB Device Disconnected  
Vendor ID: 22d9  
Product ID: 2764
```

The notification duration is set to three seconds. The notification code does not use `threaded=True`, which was removed during testing because repeated notifications were not working reliably.

14. Testing Procedure

1. Start the monitor using `python usb_monitor.py`.
2. Confirm that the startup messages appear in Command Prompt.
3. Connect a USB device to the laptop.
4. Verify that a [CONNECTED] event appears in Command Prompt.
5. Verify that a USB Device Connected Windows notification appears.
6. Remove the USB device.
7. Verify that a [DISCONNECTED] event appears in Command Prompt.
8. Verify that a USB Device Disconnected Windows notification appears.
9. Repeat the connect and disconnect actions to confirm that notifications continue to appear.

15. Troubleshooting During Development

The following issues were encountered and resolved during the build:

Python code block markers copied into the file: The text ````python` and ````` was accidentally included in the Python file, causing a `SyntaxError`. The markers were removed so that the file contained only Python code.

IndentationError: Some Python blocks did not have the required indentation. The code was re-entered in smaller sections to preserve Python indentation.

Repeated Windows notifications: The initial notification implementation used `threaded=True`. During testing, only the first notification was appearing reliably. The `threaded=True` parameter was removed and the notification duration was set to three seconds.

Package verification: `win10toast` was checked with `pip`. The system confirmed that the package was already installed.

16. Useful Commands Used

Open the Python file

```
notepad usb_monitor.py
```

List files and folders

```
dir
```

Move into the src directory

```
cd src
```

Install/verify win10toast

```
pip install win10toast
```

Install/verify win10toast with Python

```
python -m pip install win10toast
```

Run the monitoring script

```
python usb_monitor.py
```

Stop the running monitor

```
Ctrl + C
```

17. How the Monitoring Works

The script does not wait for a Windows USB event directly. Instead, it periodically scans the USB device list. At startup, the current device list is stored as `previous_devices`. Every two seconds, the script creates a new `current_devices` list and compares the two sets.

```
connected = current_devices - previous_devices
disconnected = previous_devices - current_devices
```

If a device exists in `current_devices` but not in `previous_devices`, it is treated as a new connection. If it existed in `previous_devices` but is no longer present in `current_devices`, it is treated as a disconnection.

18. Security Relevance

USB monitoring can support endpoint visibility and security monitoring. In an enterprise environment, similar controls can help identify unexpected removable media activity, support investigations, and provide additional context during incident response. This project is a basic monitoring implementation and is not, by itself, an enterprise-grade USB access-control solution.

19. Project Outcome

The completed tool successfully monitors USB devices on the Windows laptop, identifies connection and disconnection events using Vendor ID and Product ID, displays the events in Command Prompt, and generates repeated Windows desktop notifications for device status changes.

20. Final Workflow Summary

1. Open the USB-Device-Monitor project folder.
2. Enter the src directory.
3. Verify usb_monitor.py exists.
4. Install or verify win10toast.
5. Add the notification logic to usb_monitor.py.
6. Save the Python file.
7. Run: `python usb_monitor.py`
8. Connect a USB device.
9. Check the Command Prompt [CONNECTED] event.
10. Check the Windows desktop notification.
11. Disconnect the USB device.
12. Check the Command Prompt [DISCONNECTED] event.
13. Check the Windows desktop notification.
14. Repeat the test to confirm repeated notifications.

21. Notes for Future Improvements

- Write connection and disconnection events to a structured log file.
- Record timestamps for every event.
- Add device names where available.
- Add an allowlist or blocklist for approved USB devices.
- Add a configurable monitoring interval.
- Integrate alerts with a centralized security monitoring platform.
- Add stronger error handling for USB backend or permission issues.

End of Document